

Chapter 5

Location-based Services

Contents

Location Services

Location Strategies

Google Play Services

Location best practices

Introduction

Mobile users take their devices with them everywhere

Location awareness can be used to offer contextual experience

Location-based services are integral to many modern mobile apps.

To ensure accurate, timely, and efficient location tracking, developers must carefully consider the appropriate strategies.

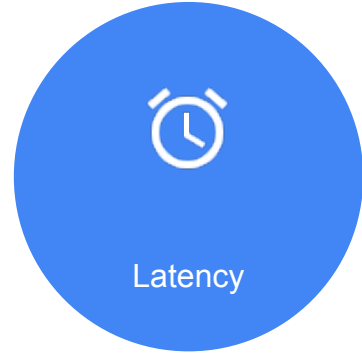
Source of location data

Two ways to obtain a user location: GPS and Network-based

Type	GPS	Network-based
Data Accuracy	High	Low
Speed	Slow	Fast
Power Consumption	High	Low
Environment	Outdoor	Indoor and outdoor

Battery Drain

Location gathering and battery drain are affected by these aspects:



Battery Drain



Accuracy

Refers to the precision of the location data.

Higher accuracy often requires more power and resources.

Battery Drain

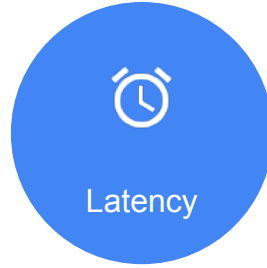


Frequency

Determines how often the device's location is updated.

A higher frequency can drain the battery faster.

Battery Drain



Latency

Measures the delay between the actual location change and the time it takes for the app to receive the updated location.

Types of accuracy

Type	Precision	Hardware Use	Power
High Accuracy	Most precise location possible	GPS	High
Balanced Power Priority	City block (100 m)	Wi-Fi or cell tower	Less
Low Power	City-level (10 km)	Wi-Fi or cell tower	Less
No power	Receives locations from other apps	None	Very minimum

Location Strategies

Global Positioning System (GPS)

High Accuracy: Provides precise location data.

Power Consumption: Can drain the battery quickly, especially when used continuously.

Use Cases: Navigation apps, outdoor activity tracking, geofencing.

Location Strategies

Network-Based Location

Lower Accuracy: Relies on cell tower and Wi-Fi triangulation.

Lower Power Consumption: Less demanding on the device's battery.

Use Cases: General location-based services, weather apps, local search.

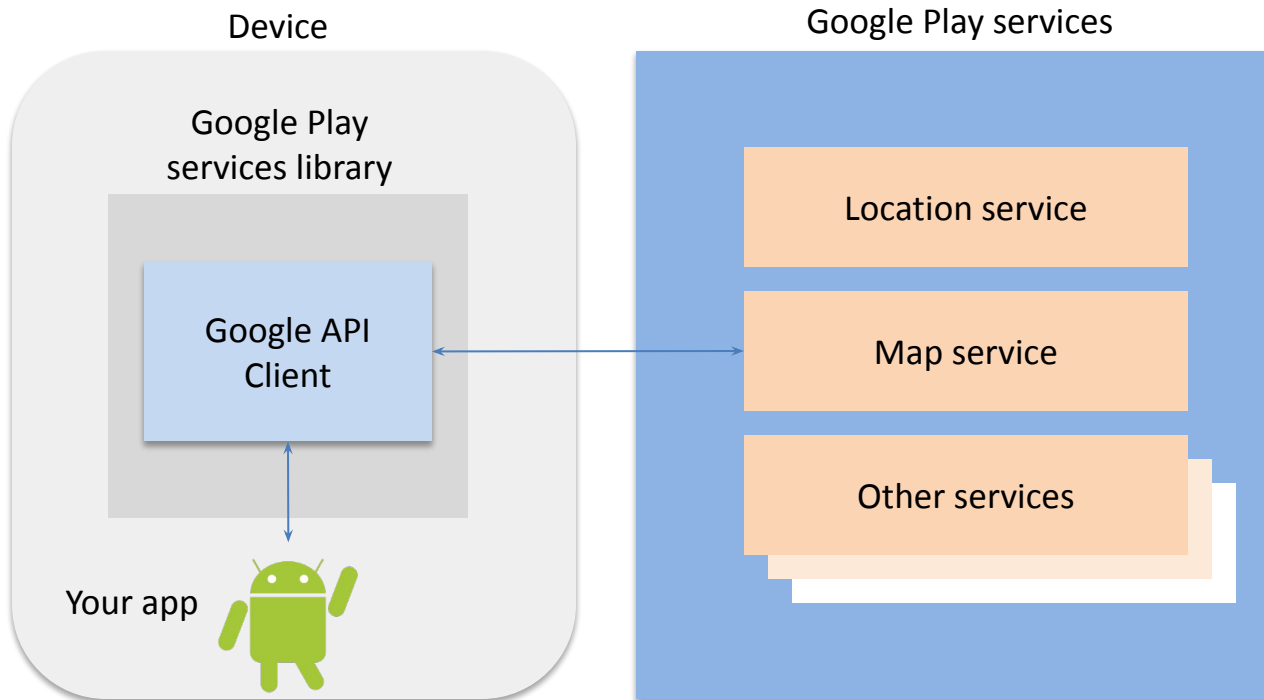
Location Strategies

Hybrid Approach:

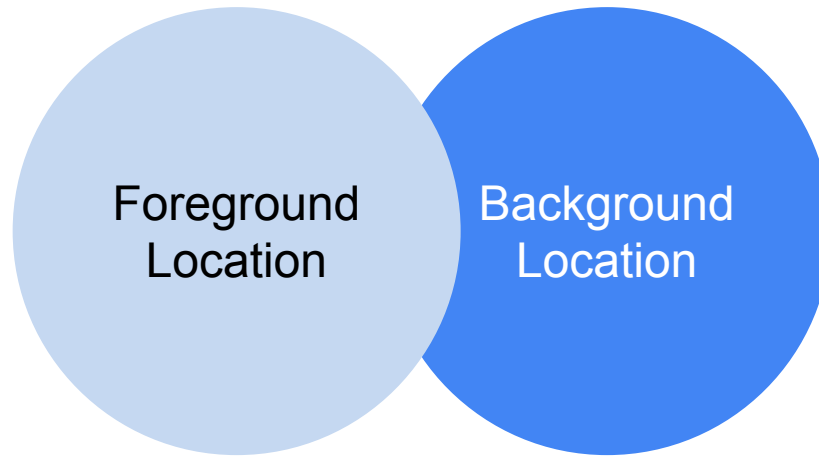
Balanced Accuracy and Power Consumption: Combines GPS and network-based location.

Use Cases: Most mobile apps that require location data.

Google Play Service



Types of Location Permission



Types of Location Permission

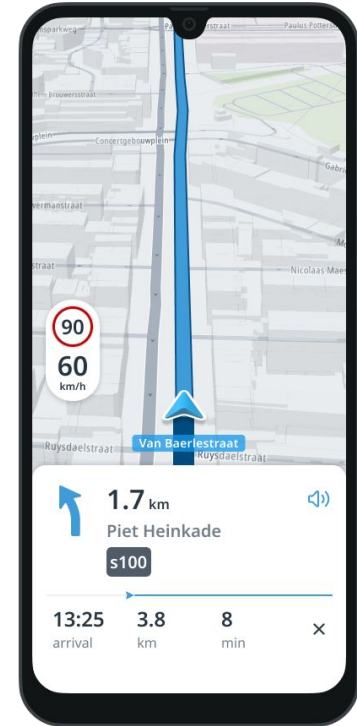
Task	Foreground Location	Background Location
Share Location	Once Predefined period	Constant
Visibility of UI	Visible	Not visible
Show persistent notification	Yes	No

Foreground Location

Examples

Within a navigation app, a feature allows users to get turn-by-turn directions.

Within a messaging app, a feature allows users to share their current location with another user.



Background Location

Examples

Within a family location sharing app, a feature allows users to continuously share location with family members.

Within an IoT app, a feature allows users to configure their home devices such that they turn off when the user leaves their home and turn back on when the user returns home.



Google Play Service

Insert permission to manifest file:

Permission	Description
Foreground - Coarse Location	Allows an app to access approximate location. Returns a location with an accuracy approximately equivalent to a city block.
Foreground - Fine Location	Allows an app to access precise location.
Background	For Android 10 (API 29) and above

Google Play Service

In the context of Android, in the Manifest file:

```
<manifest ... >
<!-- Always include this permission →
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

<!-- Include only if your app benefits from precise location access. -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

<!-- Required only when requesting background location access on
    Android 10 (API level 29) and higher. -->
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />

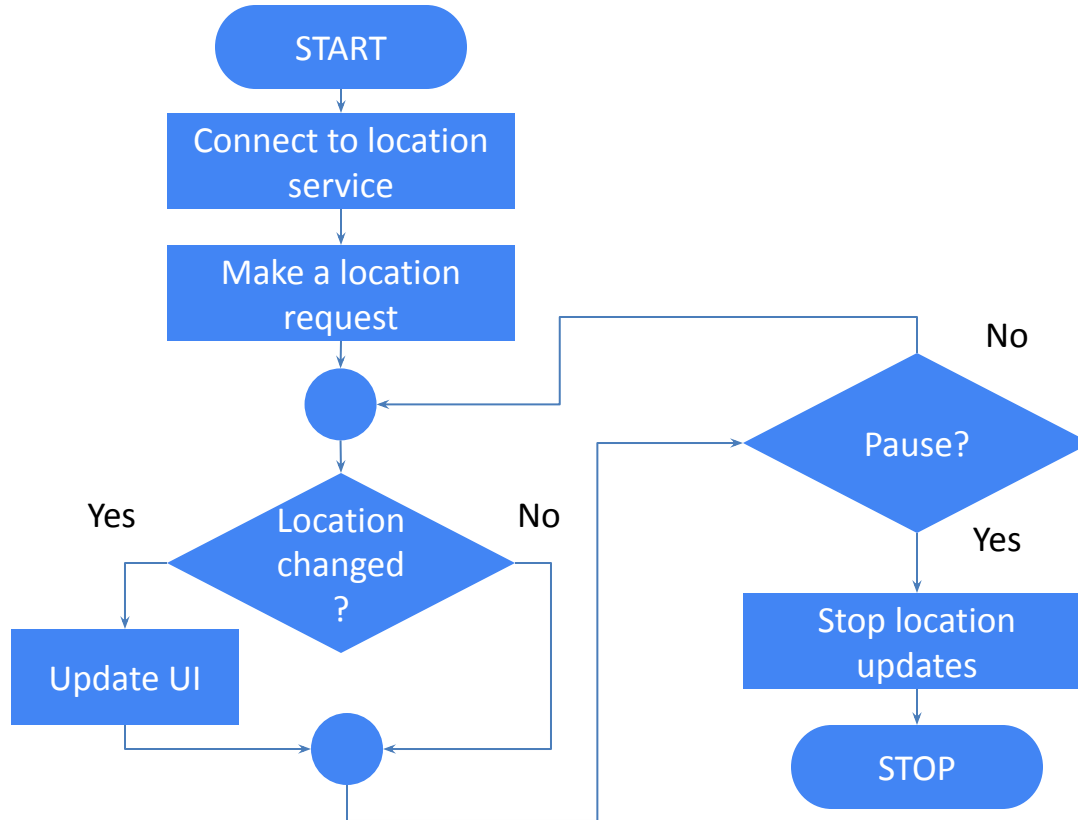
</manifest>
```

Google Play Service

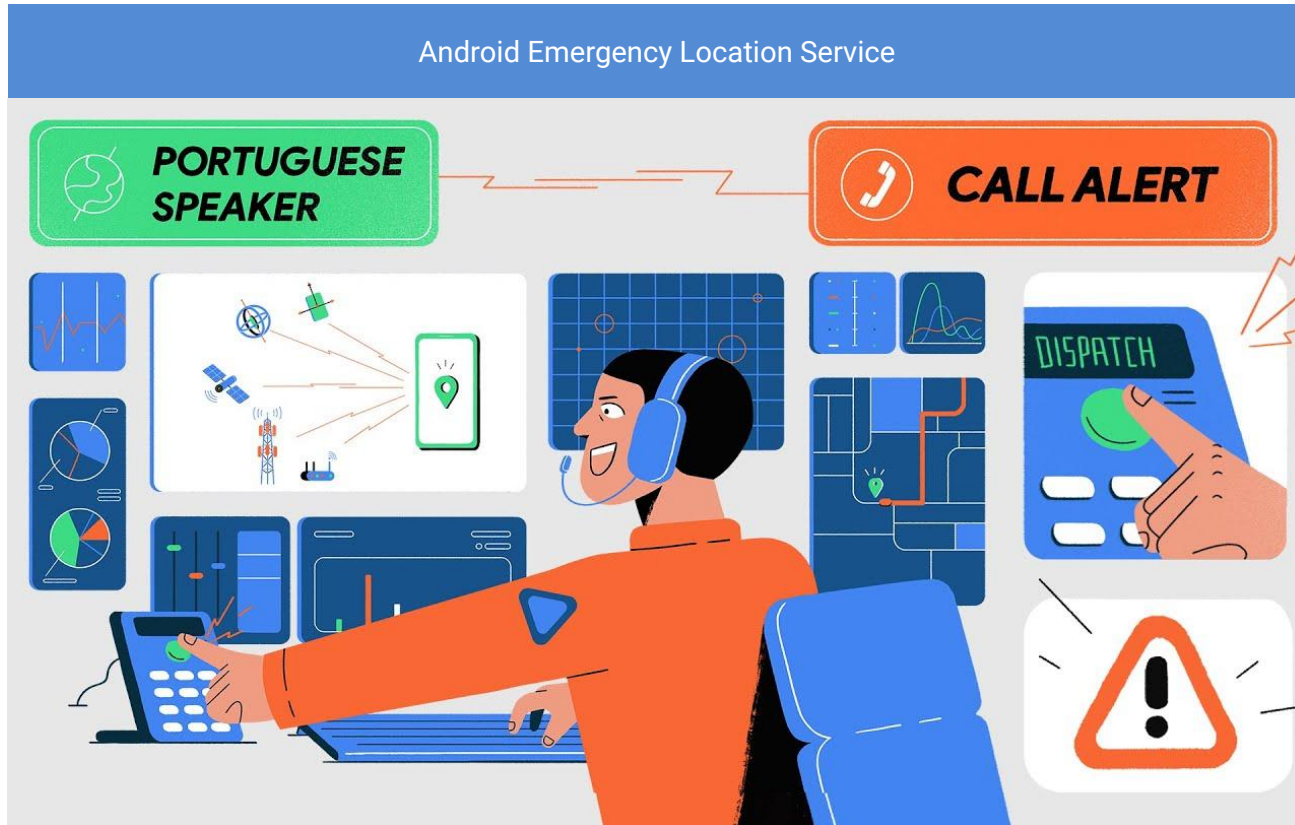
For foreground service type, insert this in the Manifest file within the application tag:

```
<!-- Recommended for Android 9 (API Level 28) and Lower. -->
<!-- Required for Android 10 (API Level 29) and higher. -->
<service
    android:name="MyNavigationService"
    android:foregroundServiceType="location">
    <!-- Any inner elements would go here. -->
</service>
```

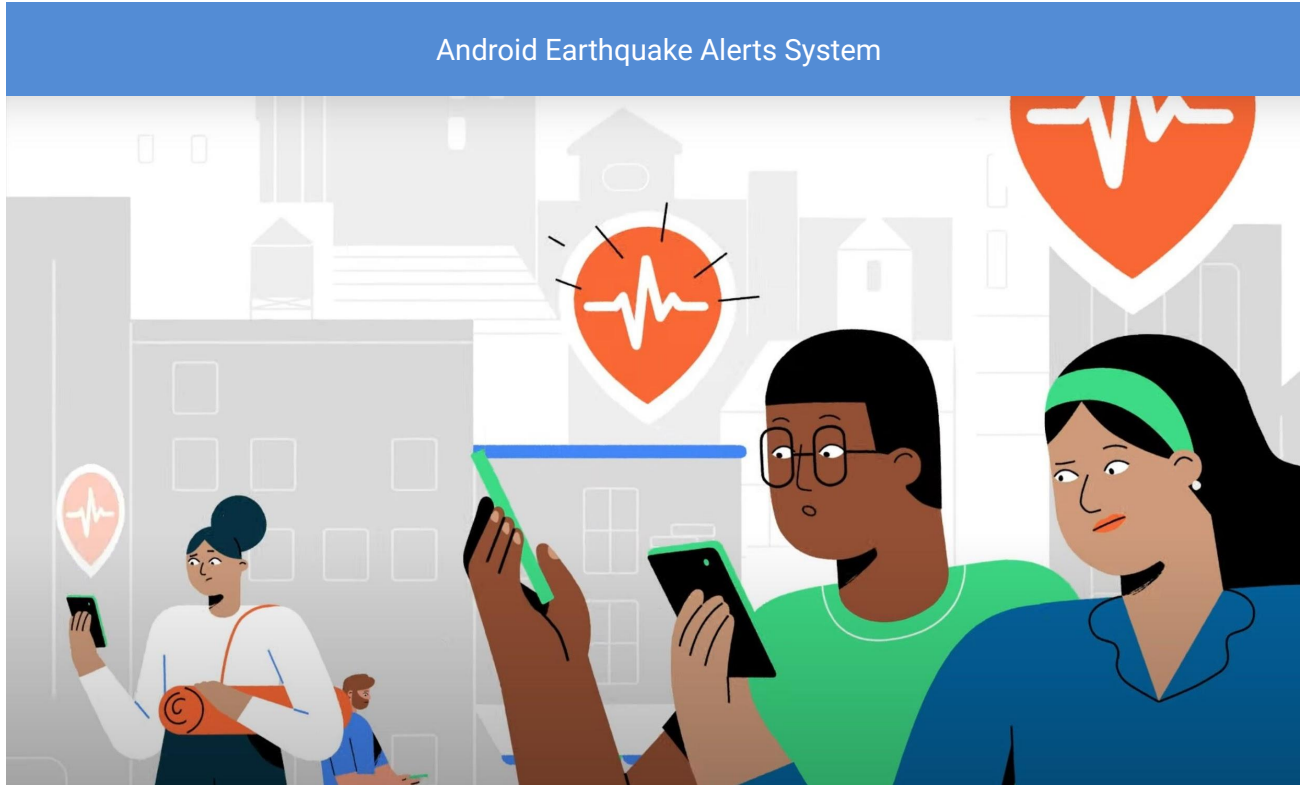
Location Updates



New Android Location Service



New Android Location Service

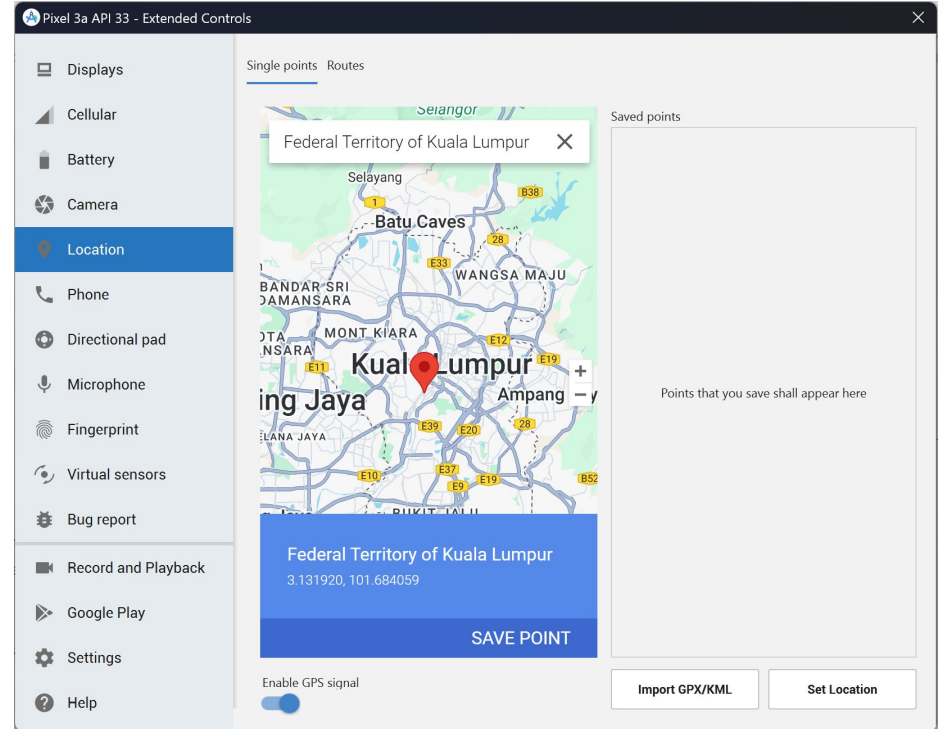


Mock Location Data

Use mock location for testing purposes

Providing mock location data by injecting GPS location data

Using Dalvik Debug Monitor Server to set location to AVD



Location Services

Location Services

1. Geocoding - convert address into geographic coordinates (latitude and longitude) or the reverse
2. Geolocation - determine current location
3. Map - display maps and, markers

Geocoding and Geolocation

Add the required dependencies, the `geolocator` and `geocoding` packages to your `pubspec.yaml` file.

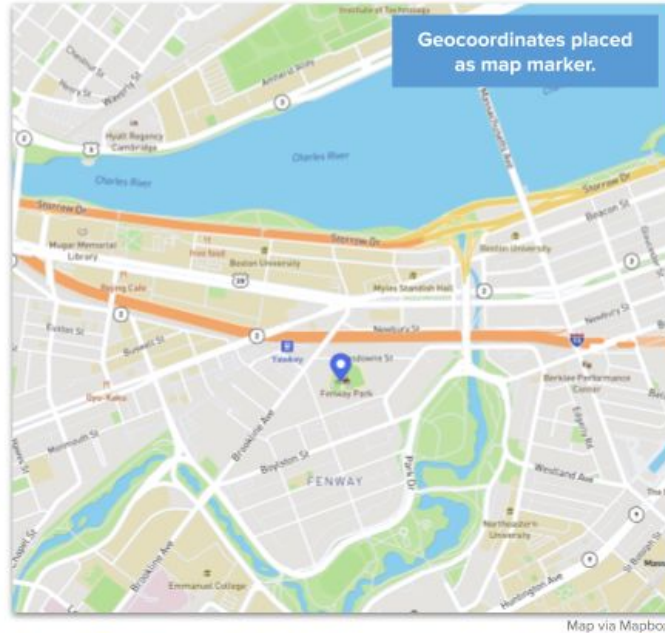
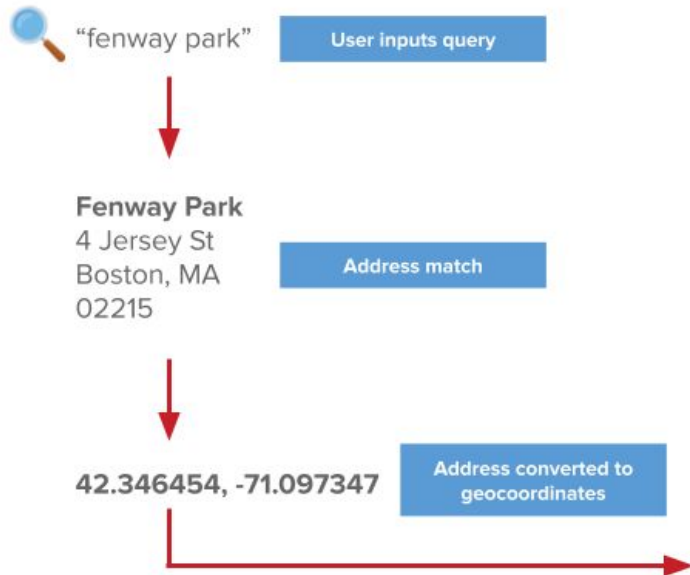
```
dependencies:
```

```
  geolocator: ^[version code]
```

```
  geocoding: ^[version code]
```

Geocoding

Geocoding = converting a geographic location to an address



Geocoding

- Geocoding APIs can compensate for ambiguous addresses — ones that are misspelled or inaccurate.
- For example, street addresses can change; address coordinates won't.
- Geocoding integrates and aligns address information with geographic codes to verify the address.

Geocoding

Possible errors:

- No location data provided
- Invalid latitude or longitude used
- No geocoder available
- No address found

Geocoding

Dependency: geocoding

```
import 'package:geocoding/geocoding.dart';
```

Geocoding

```
Future<void> getAddressFromCoordinates(double latitude, double longitude) async {  
  try {  
    List<Placemark> placemarks = await placemarkFromCoordinates(latitude, longitude,);  
  
    if (placemarks.isNotEmpty) {  
      Placemark placemark = placemarks.first;  
      String address = '${placemark.street}, ${placemark.locality},  
                        ${placemark.country}';  
      print(address);  
    } else {  
      print('Could not obtain address');  
    }  
  } catch (e) {  
    print('Error: $e');  
  }  
}
```


Geolocation

Use the device location detection hardware to determine Current Location

```
import 'package:geolocator/geolocator.dart';

...

Future<Position> getCurrentPosition() async {
  Position position = await Geolocator.getCurrentPosition(
    desiredAccuracy: LocationAccuracy.best,
  );
  return position;
}
```

Location Services

Android

Add the following lines below to your gradle.properties file:

```
android.useAndroidX=true
```

```
android.enableJetifier=true
```

Location Services

Android

Make sure to set the `compileSdkVersion` in your `android/app/build.gradle` file to `>= 33`.

```
android {  
    compileSdkVersion 33  
  
    // ...  
}
```

Location Services

Android

Add either the ACCESS_FINE_LOCATION (precise location) or the ACCESS_COARSE_LOCATION (results equal to about a city block) permission your android/app/src/main/AndroidManifest.xml file.

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

Location Services

iOS

Add the following lines inside `ios/Runner/Info.plist` to access the device's location

```
<key>NSLocationWhenInUseUsageDescription</key>  
<string>This app needs access to location when open.</string>  
<key>NSLocationAlwaysUsageDescription</key>  
<string>This app needs access to location when in the background.</string>
```

Map

Map

There are several map service providers that you can use to integrate maps into your Flutter app:

1. Google Maps
2. Mapbox
3. Apple Maps
4. Here
5. Azure Maps
6. OpenStreetMap
7. TomTom

Google Maps

To enable Google Maps in your Flutter app, you'll need the `google_maps_flutter` dependency:

```
dependencies:
```

```
  google_maps_flutter: ^2.0.0
```


Google Maps

Create a new Google Cloud Platform project.

Enable the Google Maps Platform API.

Create an API key and restrict its usage to your app's package name and SHA-1 fingerprint.

Google Maps

Android

In the AndroidManifest.xml file, insert:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application ...>

        <meta-data android:name="com.google.android.geo.API_KEY"
            android:value="YOUR_API_KEY_HERE"/>

    <activity ...>
```

Google Maps

iOS

In your Info.plist file (iOS), add the API key to the GoogleMapsApiKey key:

```
<key>GoogleMapsApiKey</key>  
<string>YOUR_API_KEY</string>
```

Google Maps

Import the necessary packages:

```
import 'package:flutter/material.dart'; import  
'package:google_maps_flutter/google_maps_flutter.dart';
```

Google Maps

Create a GoogleMap Widget

```
class _GoogleMapExampleState extends State<GoogleMapExample> {  
  
  late GoogleMapController _controller;  
  
  static const _initialCameraPosition = CameraPosition(  
    target: LatLng(37.42796133580664, -122.085749655962),  
    zoom: 14.4746,  
  );  
  
  void _onMapCreated(GoogleMapController controller) {  
    _controller = controller;  
  }  
  ...  
}
```

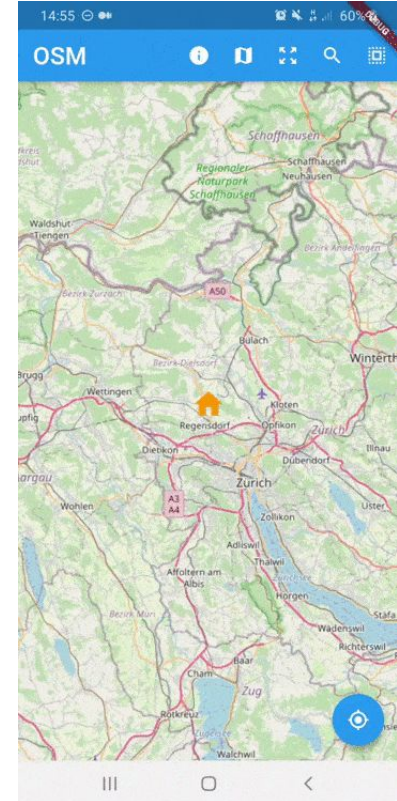
Google Maps

Create a GoogleMap Widget

```
...  
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: Text('Google Maps'),  
    ),  
    body: GoogleMap(  
      mapType: MapType.hybrid,  
      initialCameraPosition: _initialCameraPosition,  
      onMapCreated: _onMapCreated,  
    ),  
  );  
}
```

OpenStreetMap

OpenStreetMap (abbreviated OSM) is an online map that uses an open geographic database, updated and maintained by a community of volunteers via open collaboration.



OpenStreetMap

Use the Flutter map package:

```
flutter_map
```

Open access, no account, no fees

OpenStreetMap

```
class _OSMFlutterMapState extends State<OSMFlutterMap> {  
  @override  
  Widget build(BuildContext context) {  
    return FlutterMap(  
      options: const MapOptions(  
        initialZoom: 10,  
        initialCenter: LatLng(3.140853, 101.693207),  
      ),  
      children: [  
        TileLayer(  
          urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',  
          userAgentPackageName: 'com.example.demo_openscreeetmap',  
        ),  
      ],  
    );  
  }  
}
```

Location best practices

Location best practices

1. Remove location updates
2. Set timeouts
3. Batch requests
4. Passive location updates

Location best practices

1. Remove location updates

Use `Geolocator.removeListener()`: When you no longer need to track the user's location, remove the listener to conserve battery and resources.

Check App State: Only request location updates when the app is actively using them.

User Interaction: Trigger location updates based on user actions, such as tapping a button or opening a specific screen.

Location best practices

2. Set Timeouts

Reasonable Timeouts: Set appropriate timeouts for location updates to prevent excessive battery drain.

Dynamic Timeouts: Adjust timeouts based on the app's current state and user activity.

Background Mode Timeouts: If using background location updates, consider setting longer timeouts to balance accuracy and power consumption.

Location best practices

3. Batch Requests

Combine Multiple Requests: If your app needs multiple location updates, combine them into a single request to reduce network overhead and battery usage.

Throttle Requests: Limit the frequency of location updates, especially when the user is not actively using the location-based features.

Location best practices

4. Passive Location Updates

To minimize battery consumption and improve user experience, it's often beneficial to rely on system-provided location data rather than continuously tracking the user's location.

Find Out More

Google Play Services, <https://developers.google.com/android/guides/overview>

Location Request,
<https://developers.google.com/android/reference/com/google/android/gms/location/LocationRequest>

Current Location,
<https://medium.com/@fernandoPTR/how-to-get-users-current-location-address-in-flutter-geolocator-geocoding-be563ad6f66a>